



TBT (Tick-By-Tick) WebSocket Feed - Integration Guide

Audience: Subscriber / consumer teams integrating with the TBT WebSocket feed **Status:** Draft (test environment) **Last updated:** 2026-06-12

1. Abstract

This document describes the TBT (Tick-By-Tick) WebSocket feed strictly from a subscriber's point of view: how to connect, what to send, and what you get back. It does not cover server internals.

It builds on the earlier "Iris Documentation" PDF and the standalone `client.js` shared with the web team, and covers the current `tbt-test` URL, the subscribe / unsubscribe / pause / resume messages, the optional `requestId` field, subscription and mode behaviour, limits, and known edge cases.

2. Overview

A single WebSocket connection gives you streaming market data:

- **You send** small JSON control messages (subscribe / unsubscribe / pause / resume).
- **You receive** market-data updates as **binary frames**, one frame per tick per instrument.

The server transmits binary, not JSON. The previously shared `client.js` is the reference client: it opens the connection, sends the subscribe message, and **decodes each binary frame into a plain object** (the JSON shapes shown in Section 10). You can use `client.js` as-is, or port its parser to your language using the byte layouts in Section 10.

3. Prerequisites

- A WebSocket client capable of:
 - Sending **text** frames (your JSON control messages).
 - Receiving **binary** frames (market data).
 - Replying to WebSocket **ping** control frames with **pong** (standard libraries such as `ws` for Node, the browser `WebSocket`, Go `gorilla/websocket`, and Python `websockets` do this automatically).
- A `clientId` issued to you (for example `X04455`).
- A JWT `token` (see Section 5; currently not enforced on test, but build for it).
- Ability to parse **little-endian** fixed-width binary (int32, int64, uint32, float64, fixed-length byte arrays) if you write your own decoder.
- **Network connectivity to the host machine.** The feed is deployed on an AWS machine. Your client must be able to reach that machine on **port 8085**. For current testing we run `client.js` from a server that has outbound access to port 8085 on the AWS host where the feed is deployed.

Make sure your environment (firewall / security group / VPN) allows that path before integrating.

4. Connection endpoint

Current test URL:

```
wss://tbt-test.choiceindia.com:8085/v1/websocket?clientId=<CLIENT_ID>&token=<JWT_TOKEN>
```

Example:

```
wss://tbt-test.choiceindia.com:8085/v1/websocket?clientId=X04455&token=<JWT Token>
```

Notes:

- Scheme is `wss://` (TLS), port **8085**, path `/v1/websocket`.
- This replaces the old `ws://172.23.7.4:8085/...` address from the original PDF.

Query parameters

Parameter	Required today	Required by design	Notes
<code>clientId</code>	Recommended	Yes	Identifies you; drives the per-user connection limit (Section 6).
<code>token</code>	No (disabled)	Yes	JWT. Auth is currently disabled on test, so a connection without a token succeeds today. This will be re-enabled (Section 5).

5. Authentication (currently disabled on test)

Authentication is switched off in the test environment right now, so you can connect without a valid `token`. Plan for it to be turned back on. When enabled, the JWT is validated during the connection handshake, before the WebSocket upgrade. Failures return an HTTP **401** with a message such as:

- `clientId` is required
- `token` is required
- `invalid token` (bad signature or unparseable)
- `invalid issuer` (issuer is not `https://sso.choiceindia.com`)
- `expired token` (the `exp` claim is in the past)

Edge case: auth failures occur at the HTTP handshake, so you get an HTTP 401 response, not a WebSocket close. Handle a failed handshake separately from a post-connect disconnect.

6. Limits

Limit	Value	Scope	Behaviour on breach
Connections per <code>clientId</code>	10	Per user	Upgrade rejected with HTTP 429 <code>{"error": "Too Many Requests: maximum connections exceeded"}</code> .
Subscriptions per connection	1000	Per connection	Combined across all three modes (<code>touchline</code> + <code>best_five</code> + <code>full</code>). See below.

Subscription-limit breach: if a `subscribe` would push you over 1000, the **entire request is rejected** (no partial subscribe) and the server sends a **text** frame:

```
total subscriptions exceeded maxSubscriptionsPerConnection: 1000
```

This is the only validation failure that sends a message back to you. The others (Section 11) are silent.

7. Subscription and control messages (client to server)

All messages you send are **JSON text frames** with this shape:

```
{
  "action": "subscribe",
  "mode": "touchline",
  "instruments": ["1@26000", "3@1", "2@62607"],
  "requestId": "optional-correlation-id"
}
```

Field reference

Field	Type	Required?	Notes
<code>action</code>	string	Required	One of <code>subscribe</code> , <code>unsubscribe</code> , <code>pause</code> , <code>resume</code> . Unknown values are ignored.
<code>mode</code>	string	Required for subscribe/unsubscribe	One of <code>touchline</code> , <code>best_five</code> , <code>full</code> . Ignored for <code>pause</code> / <code>resume</code> .
<code>instruments</code>	string[]	Required (non-empty) for subscribe/unsubscribe	Each entry <code>"<SegmentID>@<ExchangeToken>"</code> (Section 8). Ignored for <code>pause</code> / <code>resume</code> .

Field	Type	Required?	Notes
requestId	string	Optional / not necessary	A correlation id for tracing only. If omitted, the server generates one internally. It has no effect on the data you receive and is never sent back (responses are binary and carry no request id). Send one only if you want support to be able to find your specific request in the logs.

Short version of what is necessary: `action` always; `mode` plus a non-empty `instruments` for subscribe/unsubscribe; nothing else. `requestId` is optional and changes nothing about the feed. For `pause/resume`, only `action` matters.

7.1 Subscribe

```
{ "action": "subscribe", "mode": "full", "instruments": ["2@62607"] }
```

On acceptance, if a recent value is available you immediately receive a snapshot frame for each newly-subscribed instrument (so you get current state without waiting for the next tick), then live updates follow.

7.2 Unsubscribe

```
{ "action": "unsubscribe", "mode": "full", "instruments": ["2@62607"] }
```

`mode` must match the mode the instrument is currently subscribed under. Unsubscribing something you are not subscribed to is a harmless no-op.

7.3 Pause / Resume

```
{ "action": "pause" }
{ "action": "resume" }
```

- `pause` stops market-data frames for this connection. The connection stays alive (pings continue) and your subscriptions are retained.
- `resume` restarts market-data frames for your existing subscriptions.
- `mode` and `instruments` are not required and are ignored.
- Pausing while already paused (or resuming while not paused) is a harmless no-op.

Pause edge case: frames produced while you are paused are **dropped, not buffered**. On `resume` you receive fresh ticks going forward, not a backlog.

8. Instrument identifier format

Each instrument is a string: `<SegmentID>@<ExchangeToken>`

- Validated against the pattern `^\d+@\d+$` (digits, one `@`, digits). Anything else is rejected (Section 11).
- `SegmentID` is the numeric segment code (table below).
- `ExchangeToken` is the exchange's numeric token. For indices, use the index token as provided by the exchange (for example `1@26000` for NIFTY 50).

Segment IDs

SegmentID	Segment
1	NSE Cash (NSE_CM)
2	NSE F&O (NSE_FNO)
3	BSE Cash (BSE_CM)
4	BSE F&O (BSE_FNO)
5	MCX F&O (MCX_FNO)
6	MCX Cash (MCX_CM)
13	NSE Currency (NSE_CD)
14	NSE Commodity (NSE_CO)

Examples: `1@26000` (NSE Cash, token 26000), `2@62607` (NSE F&O), `5@562396` (MCX F&O).

9. Modes

Mode	Depth	Binary frame size
<code>touchline</code>	Best bid/ask + last trade price	94 bytes
<code>best_five</code>	5 bid + 5 ask levels	214 bytes
<code>full</code>	20 bid + 20 ask levels + full stats / OI	631 bytes

You identify the mode of an incoming frame by its **byte length** (94 / 214 / 631). There is no type header. Any other length is unexpected and should be logged and skipped.

10. What you receive (market-data frames)

Frames arrive as **binary, little-endian**, fixed-size structures with no padding between fields. The client decodes each frame into an object. For each mode below you get: the decoded object (what your code sees after parsing) and the exact byte layout (what you parse).

Common conventions

- **Endianness:** little-endian for all multi-byte numbers.
- **Prices are raw integers.** Divide by `price_divisor` (a `uint32` present in every frame) to get the real price. Example: raw `2398000`, divisor `100`, gives `23980.00`.
- **token** is a fixed **50-byte** UTF-8 field, null-padded. Trim trailing `\0` to get the string.
- **segment** is the numeric SegmentID (Section 8).
- A **level** (one price row) is **12 bytes**: `int32 price`, `int32 quantity`, `int32 total_orders`.
- Timestamps: `last_update_timestamp` and `last_trade_timestamp` are **nanoseconds** since epoch (int64). `timestamp` (full mode only) is the server send time in **milliseconds** (int64).

10.1 Touchline

Expected response (actual live capture, `subscribe -> touchline -> 2@50585`):

```
{
  "best_buy": 7800,
  "best_sell": 7820,
  "last_trade_price": 7820,
  "prev_close_price": 3885,
  "volume": 127786230,
  "total_bid_qty": 3867760,
  "total_offer_qty": 2619890,
  "last_update_timestamp": 1781246197202524000,
  "price_divisor": 100,
  "segment": 2,
  "token": "50585"
}
```

Real price = raw / `price_divisor`, so `best_buy` here is `7800 / 100 = 78.00`, `best_sell` is `78.20`, and `last_trade_price` is `78.20`.

Byte layout (94 bytes):

Offset	Size	Type	Field
0	4	int32	<code>best_buy</code> (raw)
4	4	int32	<code>best_sell</code> (raw)
8	4	int32	<code>last_trade_price</code> (raw)
12	4	int32	<code>prev_close_price</code> (raw)
16	4	int32	<code>volume</code> (total trade qty)
20	4	int32	<code>total_bid_qty</code>
24	4	int32	<code>total_offer_qty</code>
28	8	int64	<code>last_update_timestamp</code> (ns)
36	4	uint32	<code>price_divisor</code>
40	4	int32	<code>segment</code>
44	50	bytes	<code>token</code> (null-padded)

Offset	Size	Type	Field
94			total

10.2 Best Five

Expected response (actual live capture, `subscribe` -> `best_five` -> `2@50585`):

```
{
  "buyLevels": [
    { "price": 7850, "quantity": 2925, "total_orders": 12 },
    { "price": 7845, "quantity": 4030, "total_orders": 14 },
    { "price": 7840, "quantity": 5590, "total_orders": 17 },
    { "price": 7835, "quantity": 4615, "total_orders": 16 },
    { "price": 7830, "quantity": 3380, "total_orders": 12 }
  ],
  "sellLevels": [
    { "price": 7875, "quantity": 3250, "total_orders": 10 },
    { "price": 7880, "quantity": 4355, "total_orders": 13 },
    { "price": 7885, "quantity": 3705, "total_orders": 11 },
    { "price": 7890, "quantity": 3770, "total_orders": 10 },
    { "price": 7895, "quantity": 4420, "total_orders": 7 }
  ],
  "last_trade_price": 7855,
  "last_trade_timestamp": 1781246233378408400,
  "prev_close_price": 3885,
  "volume": 127899720,
  "total_bid_qty": 3913845,
  "total_offer_qty": 2605460,
  "last_update_timestamp": 1781246233485725200,
  "price_divisor": 100,
  "segment": 2,
  "token": "50585"
}
```

`buyLevels` / `sellLevels` always contain exactly 5 entries, ordered best-to-worst (highest bid first, lowest ask first).

Note: the reference `client.js` parser emits the offer-quantity key as `total_offer_qty` (a typo in the client, not the feed). It is the same value documented as `total_offer_qty` in the byte layout below. Rename it in your own parser if you prefer the correct spelling.

Byte layout (214 bytes):

Offset	Size	Type	Field
0	60	level[5]	<code>buyLevels</code> (5 x 12-byte level)
60	60	level[5]	<code>sellLevels</code> (5 x 12-byte level)
120	4	int32	<code>last_trade_price</code> (raw)
124	8	int64	<code>last_trade_timestamp</code> (ns)
132	4	int32	<code>prev_close_price</code> (raw)

Offset	Size	Type	Field
136	4	int32	volume
140	4	int32	total_bid_qty
144	4	int32	total_offer_qty
148	8	int64	last_update_timestamp (ns)
156	4	uint32	price_divisor
160	4	int32	segment
164	50	bytes	token (null-padded)
214			total

Note: the old PDF's Best Five example omitted `prev_close_price`, `volume`, `total_bid_qty`, `total_offer_qty` and `last_update_timestamp`. The current frame includes them, which is why it is 214 bytes.

10.3 Full

Expected response (actual live capture, `subscribe -> full -> 2@50585`):


```

{
  "buyLevels": [
    { "price": 7695, "quantity": 1560, "total_orders": 5 },
    { "price": 7690, "quantity": 3640, "total_orders": 14 },
    { "price": 7685, "quantity": 3770, "total_orders": 12 },
    { "price": 7680, "quantity": 5330, "total_orders": 14 },
    { "price": 7675, "quantity": 3640, "total_orders": 9 },
    { "price": 7670, "quantity": 5720, "total_orders": 11 },
    { "price": 7665, "quantity": 6500, "total_orders": 13 },
    { "price": 7660, "quantity": 5135, "total_orders": 9 },
    { "price": 7655, "quantity": 5135, "total_orders": 13 },
    { "price": 7650, "quantity": 4875, "total_orders": 11 },
    { "price": 7645, "quantity": 4615, "total_orders": 9 },
    { "price": 7640, "quantity": 2080, "total_orders": 8 },
    { "price": 7635, "quantity": 1235, "total_orders": 6 },
    { "price": 7630, "quantity": 455, "total_orders": 4 },
    { "price": 7625, "quantity": 2665, "total_orders": 8 },
    { "price": 7620, "quantity": 325, "total_orders": 2 },
    { "price": 7615, "quantity": 650, "total_orders": 3 },
    { "price": 7610, "quantity": 325, "total_orders": 2 },
    { "price": 7605, "quantity": 975, "total_orders": 4 },
    { "price": 7600, "quantity": 50895, "total_orders": 126 }
  ],
  "sellLevels": [
    { "price": 7710, "quantity": 65, "total_orders": 1 },
    { "price": 7715, "quantity": 1170, "total_orders": 4 },
    { "price": 7720, "quantity": 3445, "total_orders": 10 },
    { "price": 7725, "quantity": 3900, "total_orders": 11 },
    { "price": 7730, "quantity": 2015, "total_orders": 7 },
    { "price": 7735, "quantity": 2080, "total_orders": 5 },
    { "price": 7740, "quantity": 2600, "total_orders": 6 },
    { "price": 7745, "quantity": 4615, "total_orders": 8 },
    { "price": 7750, "quantity": 5330, "total_orders": 9 },
    { "price": 7755, "quantity": 4485, "total_orders": 8 },
    { "price": 7760, "quantity": 2405, "total_orders": 7 },
    { "price": 7765, "quantity": 2665, "total_orders": 7 },
    { "price": 7770, "quantity": 1495, "total_orders": 5 },
    { "price": 7775, "quantity": 2145, "total_orders": 4 },
    { "price": 7780, "quantity": 1755, "total_orders": 4 },
    { "price": 7785, "quantity": 1495, "total_orders": 5 },
    { "price": 7790, "quantity": 1300, "total_orders": 3 },
    { "price": 7795, "quantity": 3315, "total_orders": 6 },
    { "price": 7800, "quantity": 1885, "total_orders": 8 },
    { "price": 7805, "quantity": 1950, "total_orders": 4 }
  ],
  "average_trade_price": 8606,
  "open_price": 7600,
  "close_price": 3885,
  "high_price": 11345,
  "low_price": 6775,
  "prev_close_price": 3885,
  "last_trade_price": 7700,
  "last_trade_timestamp": 1781246275009621200,
  "timestamp": 1781246275054,
  "last_trade_quantity": 130,
  "total_buy_quantity": 3894280,
  "total_sell_quantity": 2624960,
  "total_trade_quantity": 128168235,
  "total_trade_count": 825659,
  "open_interest": 15843165,
  "open_interest_high": 16740555,
  "open_interest_low": 7220005,
  "prev_open_interest": 7220005,

```

```

    "lpp_low": 4730,
    "lpp_high": 11035,
    "price_divisor": 100,
    "token": "50585",
    "segment": 2,
    "is_index": 0
}

```

buyLevels / sellLevels always contain exactly 20 entries each (all 20 shown above), ordered best-to-worst. is_index is 0, confirming a tradable instrument, so the full depth, quantities, open interest and average_trade_price are all populated. Prices are still raw: average_trade_price 8606 / 100 = 86.06, and lpp_low / lpp_high (the lower/upper circuit price band) are 47.30 / 110.35.

Byte layout (631 bytes):

Offset	Size	Type	Field
0	240	level[20]	buyLevels (20 x 12)
240	240	level[20]	sellLevels (20 x 12)
480	8	float64	average_trade_price (raw price as a float64; divide by price_divisor)
488	4	int32	open_price (raw)
492	4	int32	close_price (raw)
496	4	int32	high_price (raw)
500	4	int32	low_price (raw)
504	4	int32	prev_close_price (raw)
508	4	int32	last_trade_price (raw)
512	8	int64	last_trade_timestamp (ns)
520	8	int64	timestamp (server, ms)
528	4	int32	last_trade_quantity
532	4	int32	total_buy_quantity
536	4	int32	total_sell_quantity
540	4	int32	total_trade_quantity
544	4	int32	total_trade_count
548	4	int32	open_interest
552	4	int32	open_interest_high
556	4	int32	open_interest_low
560	4	int32	prev_open_interest
564	4	int32	lpp_low (raw)
568	4	int32	lpp_high (raw)
572	4	uint32	price_divisor
576	50	bytes	token (null-padded)

Offset	Size	Type	Field
626	4	int32	segment
630	1	uint8	is_index (0/1)
631			total

Field-order caution: in full mode the order is Open, Close, High, Low (Close comes before High). Decode strictly by offset, not by the order fields appear in the JSON example.

10.4 Reference decoder

The previously shared `client.js` (Node, `ws`) is the canonical parser and matches the layouts above. It switches on `buf.length` (94 / 214 / 631) to pick the parser, reads each field with `readInt32LE` / `readBigInt64LE` / `readDoubleLE` / `readUInt32LE`, and treats any non-binary frame as a text status/error message. Use it as a port target for other languages.

11. Validation and silent rejections

With one exception, invalid subscribe requests are **rejected silently**: the request is dropped and you get no message back. Do not wait for an error frame. If you sent a malformed request, you simply will not get data.

Condition	Server behaviour	Message back to you?
<code>mode</code> not one of <code>touchline/best_five/full</code>	Subscribe rejected	No
<code>instruments</code> empty	Subscribe rejected	No
Any instrument fails <code>^\d+@\d+\$</code>	Whole request rejected (not just the bad entry)	No
Subscriptions would exceed 1000	Whole request rejected	Yes (text: <code>total subscriptions exceeded</code> <code>maxSubscriptionsPerConnection: 1000</code>)
Body is not valid JSON	Message ignored	No
<code>action</code> is unknown	Message ignored	No

Implication: validate instrument strings on your side before sending. One malformed entry rejects the entire batch with no notification.

12. Subscription behaviour and mode strength

Modes have a strength order: **touchline (weakest) < best_five < full (strongest)**. The feed keeps **one effective subscription per instrument**, at the strongest mode you requested.

- **Upgrade:** if `2@62607` is subscribed as `touchline` and you then subscribe it as `full`, it moves to `full`. You then receive only `full` frames for it.

- **Downgrade ignored:** if it is on `full` and you subscribe it as `touchline`, it stays on `full` (the weaker request is treated as already covered).
- **Same-mode duplicate:** re-subscribing in the mode it is already in is ignored.
- **Counting toward the 1000 limit:** an instrument uses exactly one slot, regardless of how many times or modes you sent it.

Practical guidance: pick the single mode you want per instrument. To change a token's mode, just subscribe it in the new mode; there is no need to unsubscribe first (though doing so is harmless).

13. Heartbeat / keep-alive

The connection is kept alive with WebSocket ping/pong control frames (not your JSON messages):

Parameter	Value	Meaning
Server ping period	~27 s	The server sends a ping roughly every 27 seconds.
Pong wait	30 s	You must respond with a pong (automatic in standard libraries). If no pong is seen within 30 s of the last one, the server closes the connection.

What you need to do:

- Use a library that auto-responds to ping with pong (the default for the libraries listed in Section 3). If you handle frames manually, you must reply to pings.
 - Optionally, if you receive no ping for about 30 s, treat the connection as stale and reconnect.
 - Pings continue even while you are paused.
-

14. Reconnection behaviour

- **No session resume.** If the connection drops and you reconnect, you start clean. You must re-send all your subscribe messages. A disconnected client's subscriptions are not remembered.
 - **Snapshot on (re)subscribe.** Each fresh subscribe gives you the latest available frame immediately (if any), so you recover current state quickly.
 - **Keep your read loop fast.** Each connection has a bounded outbound buffer. If you read too slowly and it fills, new frames are dropped for your connection rather than blocking or disconnecting you. You can silently miss ticks if you cannot keep up, so offload heavy processing off the read path.
 - **Close frames.** A clean close uses a normal WebSocket close frame; abnormal closures (network or heartbeat timeout) may arrive without a reason.
-

15. Quick start checklist

1. Confirm network connectivity to the AWS host on port 8085 (Section 3).
2. Open `wss://tbt-test.choiceindia.com:8085/v1/websocket?clientId=<ID>` (add

&token=<JWT> when auth is enabled).

3. For TLS on test, allow the non-public certificate (test only).
4. On open, send a subscribe text frame, for example
`{"action":"subscribe","mode":"touchline","instruments":["1@26000"]}`.
5. On each binary message, branch on length (94 / 214 / 631) and parse per Section 10 (little-endian).
6. Convert prices using `price_divisor`; trim the 50-byte `token`.
7. Make sure your library auto-replies to pings.
8. Handle text frames as status/error (for example the subscription-limit message).
9. On disconnect, reconnect and re-subscribe everything.

16. Glossary of fields

Field	Meaning
<code>best_buy / best_sell</code>	Best bid / best ask price (raw; divide by <code>price_divisor</code>).
<code>last_trade_price</code> (LTP)	Price of the most recent trade (raw).
<code>prev_close_price</code>	Previous session close (raw).
<code>volume / total_trade_quantity</code>	Cumulative traded quantity for the session.
<code>total_bid_qty / total_offer_qty</code>	Total quantity on the bid / offer side.
<code>total_buy_quantity / total_sell_quantity</code>	Same values in full-frame naming.
<code>average_trade_price</code> (ATP)	Volume-weighted average price (raw price as a float64; divide by <code>price_divisor</code>).
<code>open / high / low / close_price</code>	Session OHLC (raw).
<code>open_interest, _high, _low, prev_open_interest</code>	Open-interest statistics (derivatives).
<code>lpp_low / lpp_high</code>	Lower / upper price band (raw).
<code>last_trade_quantity</code>	Quantity of the most recent trade.
<code>total_trade_count</code>	Number of trades in the session.
<code>total_orders</code> (per level)	Number of orders aggregated at that price level.
<code>price_divisor</code>	Divide raw prices by this to get real prices.
<code>segment</code>	Numeric segment id (Section 8).
<code>token</code>	Exchange token string (50-byte, null-padded).
<code>is_index</code>	<code>1</code> if the instrument is an index, else <code>0</code> (full mode).
<code>last_trade_timestamp / last_update_timestamp</code>	Event time in nanoseconds.
<code>timestamp</code>	Server send time in milliseconds (full mode).

End of document (draft).